

CMP project 4

Karmukilan Somasundaram

March 2026

1 Introduction

Our goal for this project is to use the nearly free electron model to plot the electron band structure for the following weak potential

$$V(x) = A \cos \frac{2\pi x}{a'} + \frac{A}{2} \cos \frac{4\pi x}{a'} + \frac{A}{3} \cos \frac{6\pi x}{a'} \quad (1)$$

where $A=3\text{eV}$ and $a'=a/2$.

2 Band structure and Density of states,convergence and Fermi Energy

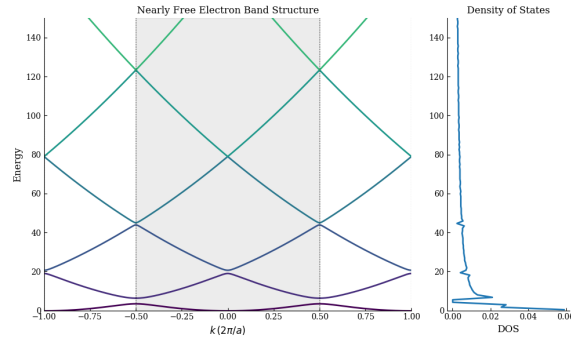


Figure 1: Band structure and density of states for $V(x)$

We can observe that the density of states distributes itself by moving away from the ground state as we increase the truncation parameter

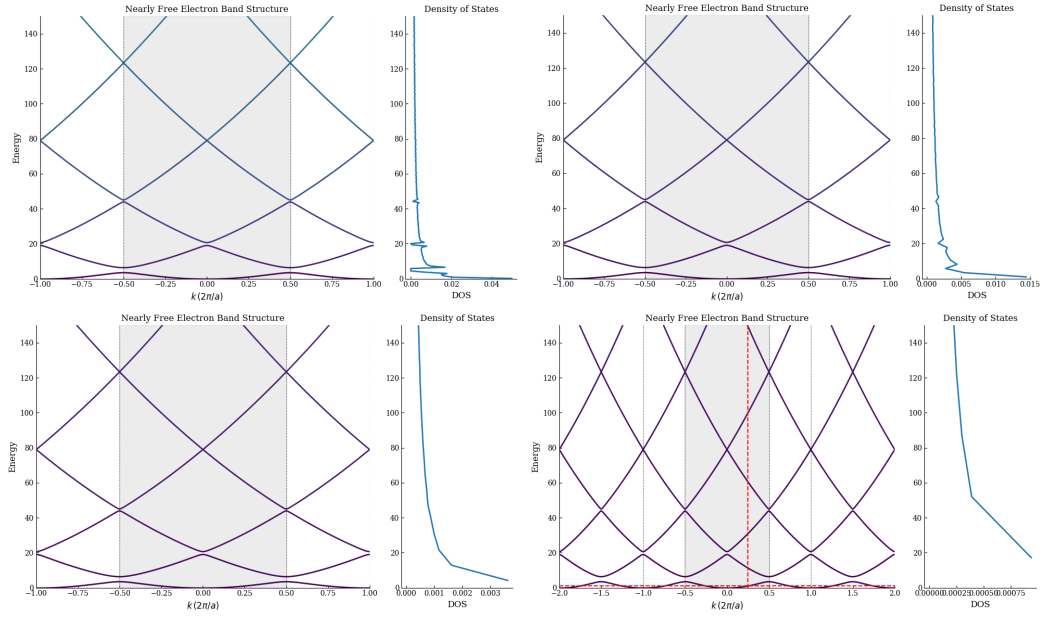


Figure 2: a.10, b.20, c.40, d.80 where the number denotes the number of oscillatory modes

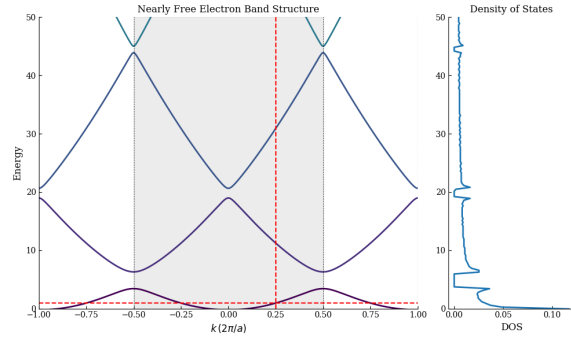


Figure 3: Fermi energy as N approaches infinity

3 2-Folding

The new weak potential is

$$V(x) = 2A \cos \frac{\pi x}{a'} + A \cos \frac{2\pi x}{a'} + \frac{A}{2} \cos \frac{4\pi x}{a'} + \frac{A}{3} \cos \frac{6\pi x}{a'} \quad (2)$$

We could now change $a'' = a'/2$ and proceed further which yields

$$V(x) = 2A \cos \frac{2\pi x}{a''} + A \cos \frac{4\pi x}{a''} + \frac{A}{2} \cos \frac{8\pi x}{a''} + \frac{A}{3} \cos \frac{12\pi x}{a''} \quad (3)$$

From this we can carry our simulation

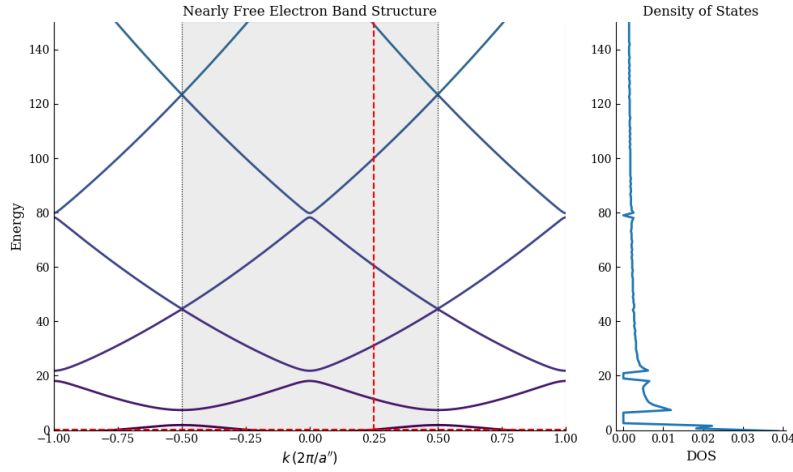


Figure 4: Folded Band structure

4 Total Energy per unit cell

This plot agrees with the fact that the asymmetry induced by the potential reduces the total energy of the system (Spontaneous Symmetry breaking)

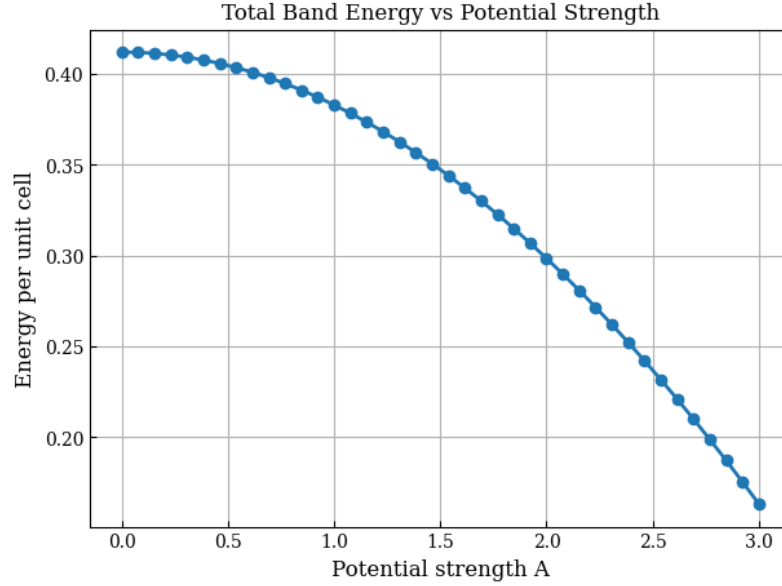


Figure 5: E_{tot} vs A

5 Python Codes

Listing 1: Band structure, Density Of States

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  plt.rcParams.update({
5      "font.family": "serif",
6      "axes.labelsize": 12,
7      "xtick.direction": "in",
8      "ytick.direction": "in",
9  })
10
11  def nfe_band_and_dos():
12
13      a_ = 1.0
14      G0 = 2*np.pi/a_
15
16      # First two Brillouin zones
17      k_min, k_max = -2, 2
18      num_k_points = 1000
19
20      n = int(input("Truncation parameter n: "))
21      v_input = input(f"Enter {n} values for V_G (comma separated): ")
22

```

```

23     v_list = [float(x.strip()) for x in v_input.split(',')]
24
25     while len(v_list) < 2*n:
26         v_list.append(0.0)
27
28     g_indices = np.arange(-n, n+1)
29     num_basis = len(g_indices)
30
31     k_vals = np.linspace(k_min*G0, k_max*G0, num_k_points)
32
33     energies_grid = []
34
35     # Construct Hamiltonian and diagonalize
36     for k in k_vals:
37
38         H = np.zeros((num_basis, num_basis))
39
40         for i in range(num_basis):
41             for j in range(num_basis):
42
43                 if i == j:
44                     H[i,j] = 0.5*(k - g_indices[i]*G0)**2
45
46                 else:
47                     m = abs(g_indices[i]-g_indices[j])
48                     if m <= len(v_list):
49                         H[i,j] = v_list[m-1]
50
51         energies_grid.append(np.linalg.eigvalsh(H))
52
53     energies_grid = np.array(energies_grid)
54
55     # =====
56     # Density of States
57     # =====
58
59     energies_flat = energies_grid.flatten()
60
61     bins = 1000
62     dos, energy_bins = np.histogram(energies_flat, bins=bins, density=True)
63
64     energy_centers = 0.5*(energy_bins[1:] + energy_bins[:-1])
65
66     # =====
67     # Plotting
68     # =====
69
70     fig = plt.figure(figsize=(10,6))
71
72     gs = fig.add_gridspec(1,2,width_ratios=[3,1])

```

```

73
74 ax_band = fig.add_subplot(gs[0])
75 ax_dos = fig.add_subplot(gs[1], sharey=ax_band)
76
77 colors = plt.cm.viridis(np.linspace(0,0.8,num_basis))
78
79 for i in range(num_basis):
80     ax_band.plot(k_vals/G0, energies_grid[:,i], color=colors[i], lw=2)
81
82 ax_band.axvspan(-0.5,0.5,color='gray',alpha=0.15)
83
84 for boundary in [-1,-0.5,0.5,1]:
85     ax_band.axvline(boundary,color='black',linestyle=':',lw=0.8)
86
87 kF = np.pi/(2*a_)
88
89 # find closest k-point
90 k_index = np.argmin(np.abs(k_vals - kF))
91
92 # first band energy
93 E_F = energies_grid[k_index,0]
94
95
96 ax_band.set_xlim(k_min,k_max)
97 ax_band.set_ylim(0,150)
98
99 ax_band.set_xlabel(r"$k\,(2\pi/a)$")
100 ax_band.set_ylabel("Energy")
101 ax_band.axhline(E_F, color='red', linestyle='--', label="Fermi_Energy")
102 ax_band.axvline(kF/G0, color='red', linestyle='--')
103
104 ax_band.set_title("Nearly_Free_Electron_Band_Structure")
105
106 # DOS plot
107 ax_dos.plot(dos, energy_centers, lw=2)
108
109 ax_dos.set_xlabel("DOS")
110 ax_dos.set_title("Density_of_States")
111
112 ax_dos.spines['top'].set_visible(False)
113 ax_dos.spines['right'].set_visible(False)
114
115 ax_band.spines['top'].set_visible(False)
116 ax_band.spines['right'].set_visible(False)
117
118 plt.tight_layout()
119 plt.show()
120
121 nfe_band_and_dos()
122

```

```

123 [style=python,caption=
124 {Total Energy}]
125 import numpy as np
126 import matplotlib.pyplot as plt
127
128 plt.rcParams.update({
129     "font.family": "serif",
130     "axes.labelsize": 12,
131     "xtick.direction": "in",
132     "ytick.direction": "in",
133 })
134
135 def nfe_band_and_dos():
136
137     a_ = 1.0
138     G0 = 2*np.pi/a_
139
140     # First two Brillouin zones
141     k_min, k_max = -2, 2
142     num_k_points = 1000
143
144     n = int(input("Truncation parameter n: "))
145     v_input = input(f"Enter {n} values for V_G (comma separated): ")
146
147     v_list = [float(x.strip()) for x in v_input.split(',')]
148
149     while len(v_list) < 2*n:
150         v_list.append(0.0)
151
152     g_indices = np.arange(-n, n+1)
153     num_basis = len(g_indices)
154
155     k_vals = np.linspace(k_min*G0, k_max*G0, num_k_points)
156
157     energies_grid = []
158
159     # Construct Hamiltonian and diagonalize
160     for k in k_vals:
161
162         H = np.zeros((num_basis, num_basis))
163
164         for i in range(num_basis):
165             for j in range(num_basis):
166
167                 if i == j:
168                     H[i,j] = 0.5*(k - g_indices[i]*G0)**2
169
170                 else:
171                     m = abs(g_indices[i]-g_indices[j])
172                     if m <= len(v_list):

```

```

173         H[i,j] = v_list[m-1]
174
175         energies_grid.append(np.linalg.eigvalsh(H))
176
177     energies_grid = np.array(energies_grid)
178
179     # =====
180     # Density of States
181     # =====
182
183     energies_flat = energies_grid.flatten()
184
185     bins = 1000
186     dos, energy_bins = np.histogram(energies_flat, bins=bins, density=True)
187
188     energy_centers = 0.5*(energy_bins[1:] + energy_bins[:-1])
189
190     # =====
191     # Plotting
192     # =====
193
194     fig = plt.figure(figsize=(10,6))
195
196     gs = fig.add_gridspec(1,2,width_ratios=[3,1])
197
198     ax_band = fig.add_subplot(gs[0])
199     ax_dos = fig.add_subplot(gs[1], sharey=ax_band)
200
201     colors = plt.cm.viridis(np.linspace(0,0.8,num_basis))
202
203     for i in range(num_basis):
204         ax_band.plot(k_vals/G0, energies_grid[:,i], color=colors[i], lw=2)
205
206     ax_band.axvspan(-0.5,0.5,color='gray',alpha=0.15)
207
208     for boundary in [-1,-0.5,0.5,1]:
209         ax_band.axvline(boundary,color='black',linestyle=':',lw=0.8)
210
211     kF = np.pi/(2*a_)
212
213     # find closest k-point
214     k_index = np.argmin(np.abs(k_vals - kF))
215
216     # first band energy
217     E_F = energies_grid[k_index,0]
218
219
220     ax_band.set_xlim(k_min,k_max)
221     ax_band.set_ylim(0,150)
222

```



```

223     ax_band.set_xlabel(r"$k\,(2\pi/a)$")
224     ax_band.set_ylabel("Energy")
225     ax_band.axhline(E_F, color='red', linestyle='--', label="Fermi_Energy")
226     ax_band.axvline(kF/G0, color='red', linestyle='--')
227
228     ax_band.set_title("Nearly_Free_Electron_Band_Structure")
229
230     # DOS plot
231     ax_dos.plot(dos, energy_centers, lw=2)
232
233     ax_dos.set_xlabel("DOS")
234     ax_dos.set_title("Density_of_States")
235
236     ax_dos.spines['top'].set_visible(False)
237     ax_dos.spines['right'].set_visible(False)
238
239     ax_band.spines['top'].set_visible(False)
240     ax_band.spines['right'].set_visible(False)
241
242     plt.tight_layout()
243     plt.show()
244
245 nfe_band_and_dos()

```